

INF115 Databases and Modelling

Obligatory Assignment 2

Deadline: 20th of April 2026, (**18:00**)

Contents

Introduction	2
Submission	2
Tips	2
Task 0: Creating the app	3
Task 1: Viewing Tables (20%)	3
Task 2: Adding Bikes (20%)	3
Task 3: Database analysis (20%)	3
Task 4: Interactions (20%)	3
Task 5: Mapping (20%)	4

Introduction

In this assignment we will create an application using Shiny for Python that uses SQLite as its backend. Continue building on the database design that you created in the obligatory assignment 1.

Install Shiny for Python if you haven't done it already.

<https://shiny.posit.co/py/get-started/install.html>

The following documentation pages may be useful:

<https://shiny.posit.co/py/api/express/>

<https://pandas.pydata.org/docs/reference/frame.html#dataframe>

Submission

You must submit your solution as a single zip-file containing:

- a **Python** file named `app.py`,
- a **SQLite** database file named `bysykke1.db`,
- additional **Python** files and **SQL** scripts are allowed,

before the deadline. Make sure that the final version of your code runs locally on your computer before delivery. Points will be deducted if there are errors when running your submission. Solutions to all tasks should be contained in **one** app. Points will be deducted if the main Python file is not called `app.py`. Comment parts of your code that need further explanation, and cite the sources of code you use. Include names of students you worked together with. Points will also be deducted for implementation in other languages than **Python** or other databases than **SQLite**.

Important! If you work together in groups, **everyone** in the group must submit the assignment on mitt.uib. In addition you must clearly state the names of all the colleagues that you worked with on the first page of the document. Late submissions will be penalised with a 10% point deduction for each late day.

Tips

Are you struggling with the tasks?

- Stay Calm!
- Recheck your notes and the slides.
- Go to your group sessions and ask your questions there.
- Ask on the Discord server.
- Search the internet as all programmers do.
- Read the Shiny documentation.

Task 0: Creating the app

Create a single app, using Shiny for Python, that implements all the functionality described in Tasks 1–5.

Task 1: Viewing Tables (20%)

The app should displays the following tables:

- (a) A table showing a bike's name, id and status. The rows should be sorted in an alphabetical order based on the bike's name.
- (b) A table showing all stations, their name, id, maximum parking spots, available parking spots and available parking spots as percentage of maximum parking spots.
- (c) A table showing how many times each subscription type was purchased, i.e. the solution to 4.3 from obligatory assignment 1.

Task 2: Adding Bikes (20%)

- (a) Update you database design to enforce the constraint that the bike status can have one of the following four values: *Active*, *Parked*, *Missing* and *Service*. Explain how you achieve this by providing SQL queries or Python code of the whole process.
- (b) Create a text input boxes, that asks for the name of the new bike and a submit button.
- (c) After the button is pressed, check that the submitted name consists only of letters. If the name is *Valid* insert the new bike into the database and set its status to *Service*. If the name contains other characters than letters, indicate to the user that the name is *Invalid*.

Task 3: Database analysis (20%)

- (a) Make sure that the bikes table from Problem 1 is updated whenever a new bike is added.
- (b) Make it possible to filter the *bikes* table based on the status, e.g. show only bikes with status *Service*.
- (c) Provide a filterable station list with the names of the available bikes at each station. One should be able to filter based on the station name.

Task 4: Interactions (20%)

In this task we are simulating some possible interaction with the database. Make sure that the tables in the app are updated when the database changes.

- (a) Create a tab "PICK UP" where a user can select a bike. Only bikes with status *Parked* can be selected. Add a *Pick up* button that changes the status of the selected bike to *Service* and updates the number of available parking spots in the *Station* table.

- (b) Add complaints to at least three different bikes that have status *Parked* in the database. You can chose the twelve possible complaint types yourself.
- (c) Create a tab "SERVICE" where a user can select a bike. Only bikes with status *Service* can be selected. Show all complaint related to the selected bike.
Add a *Service* button that services all complaints and returns the bike to a station chosen by the user. Update the database accordingly.

Task 5: Mapping (20%)

Query the *bysykket* database and produce a table that shows how many trips have started and ended on each station. In addition provide a link to each of the stations, where one can see the station's position in Openstreetmap. Given the `LATITUDE` and `LONGITUDE`, such a link would be:

`https://www.openstreetmap.org/#map=17/LATITUDE/LONGITUDE`

Example: `https://www.openstreetmap.org/#map=17/60.391270/5.325756`

Hint: set the values in the link column to

```
'<a href="https://www.openstreetmap.org/#map=17/LATITUDE/LONGITUDE">Map</a>'
```

and use `@render.table(render_links=True, escape=False)` to render the table.